

Practical-10

Name: Pavan Ankush Badak

Div: SY-A

Roll No: A-74

Branch: AI&DS

Aim

To study and implement Version Control using Git and GitHub.

Theory

Version Control System (VCS) is a tool that helps track changes in files over time. It allows multiple users to collaborate, maintain history, and revert to previous versions.

Git is a distributed version control system that tracks changes locally.

GitHub is an online platform that hosts Git repositories and enables collaboration.

Step-by-Step Procedure

1. Install Git

Download and install Git from official website.

Verify installation:

`git --version`

Screenshot 1: Git version output

```
C:\Users\pawan>git --version
git version 2.50.1.windows.1
```

2. Configure Username & Email

`git config --global user.name "Your Name"`

`git config --global user.email "your@email.com"`

```
C:\Users\pawan>git config --global user.name "PawanBadak"
C:\Users\pawan>git config --global user.email "your@email.com"

C:\Users\pawan>git config --global user.email "pawanbadak4@gmail.com"

C:\Users\pawan>cd C:\Users\pawan\PDS_Practicals
C:\Users\pawan\PDS_Practicals>git init
```

3. Create Local Repository

Go to your folder:

```
cd C:\Users\pawan\PDS_Practicals  
git init
```

Screenshot 2: Repository creation (git init output)

```
C:\Users\pawan>cd C:\Users\pawan\PDS_Practicals  
  
C:\Users\pawan\PDS_Practicals>git init  
Reinitialized existing Git repository in C:/Users/pawan/PDS_Practicals/.git/
```

4. Create Files

Create a file manually or via command:

```
echo Hello Git > file1.txt
```

5. Add Files to Staging

```
git add file1.txt
```

```
C:\Users\pawan\PDS_Practicals>echo Hello Git > file1.txt  
  
C:\Users\pawan\PDS_Practicals>git add file1.txt  
  
C:\Users\pawan\PDS_Practicals>git commit -m "First commit"  
[main 8097b16] First commit  
1 file changed, 1 insertion(+)  
create mode 100644 file1.txt
```

6. Commit Changes

```
git commit -m "First commit"
```



Screenshot 3: Commit history

```
C:\Users\pawan\PDS_Practicals>git commit -m "First commit"  
[main 8097b16] First commit  
1 file changed, 1 insertion(+)  
create mode 100644 file1.txt
```

7. Create GitHub Repository

Create repo: PDS-Practicals

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

 PawanBadak ▾

Repository name *

PDS-Practicals

✓ PDS-Practical is available.

Great repository names are short and memorable. How about [shiny-guacamole?](#)

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

3

Jumpstart your project with Copilot (optional)

Tell [Copilot cloud agent](#) what you want to build in this repository. After creation, Copilot will open a pull request with generated files - such as a basic app, starter code, or other features you describe - then request your review when it's ready.

Prompt

Describe what you want Copilot to build

0 / 30000 characters

Create repository

8. Link Local Repo to GitHub

`git remote add origin https://github.com/PawanBadak/PDS-Practicals.git`

9. Push Files to GitHub

`git branch -M main`

`git push -u origin main`



Screenshot 5: Successful push

```
C:\Users\pawan\PDS_Practicals>git branch -M main

C:\Users\pawan\PDS_Practicals>git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 487 bytes | 121.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/PawanBadak/PDS-Practicals.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

10. Display Commit Logs

`git log`



Screenshot 6: git log output

```
C:\Users\pawan\PDS_Practicals>git log
commit 8097b16a7bb9eca1877f1a1122de2ab7a331cdc3 (HEAD -> main, origin/main)
Author: PawanBadak <pawanbadak4@gmail.com>
Date:   Mon May 4 22:58:07 2026 +0530

    First commit

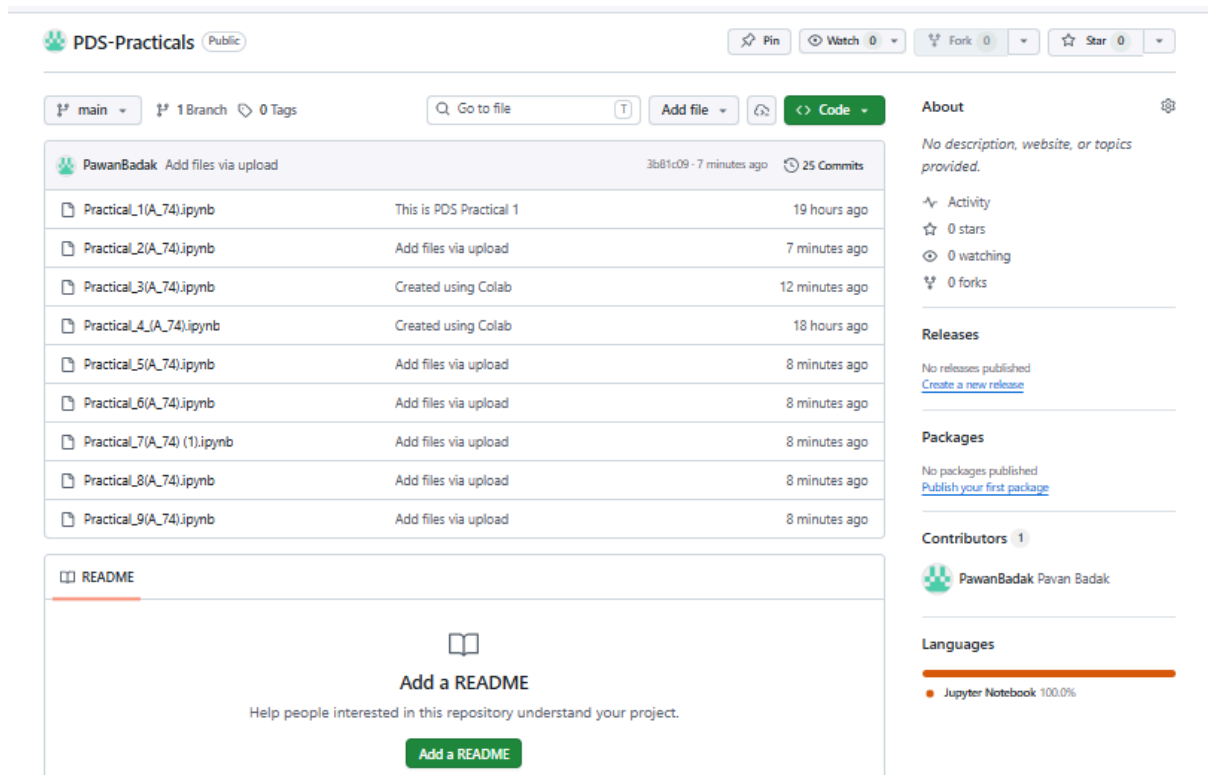
commit e2fb17d7b13ea81966670f6c1e5dcb1762511426
Author: Pavan <collegeworkspace74@gmail.com>
Date:   Mon May 4 22:21:42 2026 +0530

    Added practical files
```

11. Check Files on GitHub

Open your repo in browser.

 Screenshot 7: Files visible on GitHub



Questions & Answers

1. Difference between Git and GitHub

Git	GitHub
Software	Web platform
Works locally	Works online
Tracks changes	Hosts repositories
No internet needed	Requires internet

2. Difference between git add and git commit

- git add → Moves file to staging area
- git commit → Saves changes permanently

3. What is Staging Area in Git?

The staging area (also called the index) is an intermediate space in Git where changes are prepared before they are permanently recorded in the repository.

When a developer modifies or creates files, those changes are not immediately saved in Git's history. Instead, they must first be added to the staging area using the `git add` command. This allows the user to carefully select which changes should be included in the next commit.

The staging area acts like a preview zone, giving full control over commits. For example, if multiple files are modified, a developer can stage only specific files and commit them, leaving others for later.

Key Points:

- It is between working directory and repository
- Helps in organizing commits
- Allows selective committing of changes
- Improves version control accuracy

In simple terms:

Staging area = Place where you prepare changes before saving them permanently

4. What happens if a file is modified after committing it?

Once a file is committed in Git, it becomes part of the project's history. However, if the same file is modified again after the commit, Git does not automatically update the previous commit.

Instead, Git detects the change and marks the file as modified in the working directory.

To save these new changes, the user must repeat the process:

```
git add filename  
git commit -m "Updated file"
```

This creates a new commit, preserving both:

- The old version (previous commit)
- The new version (latest commit)

Important Concept:

Git never overwrites history. Every change creates a new version, making it easy to:

- Track changes
- Compare versions
- Revert if needed

Example:

- Commit 1 → file has "Hello"
 - Modify file → now "Hello World"
 - Commit 2 → stores updated version
-

5. Difference between git push and git pull

These commands are used to interact with remote repositories like GitHub.

Feature	git push	git pull
Function	Uploads changes	Downloads changes
Direction	Local → Remote	Remote → Local
Purpose	Share your work	Get updated code
Usage	After commit	Before starting work

6. What is Version Control? Differentiate between Centralized and Distributed VCS

Version Control

Version Control is a system that records changes to files over time so that specific versions can be recalled later. It is widely used in software development to manage source code.

It helps developers to:

- Track history of changes
- Collaborate with others
- Restore previous versions
- Avoid conflicts

Types of Version Control Systems

1. Centralized Version Control System (CVCS)

In a centralized system, there is a single central server that stores all files and their versions. Users connect to this server to access or modify files.

Features:

- One main repository
- All users depend on server
- Requires internet connection

Example:

- SVN (Apache Subversion)

Advantages:

- Easy to manage
- Simple structure

Disadvantages:

- Single point of failure (if server crashes, data may be lost)
 - No offline work
-

2. Distributed Version Control System (DVCS)

In a distributed system like Git, every user has a complete copy of the repository, including its history.

Features:

- Each user has full backup
- Works offline
- Faster operations

Example:

- Git

Advantages:

- No dependency on central server
- High speed
- Better collaboration
- Safe (data not lost easily)

Disadvantages:

- Slightly complex for beginners
- Requires understanding of commands

Difference Table

Feature	Centralized VCS	Distributed VCS
Repository	Single central server	Multiple copies
Internet	Required	Not always required
Speed	Slower	Faster
Data Safety	Risky	Safer
Example	SVN	Git